



WEB班ワークショップアジェンダ第一案

基本フロー：完成形から逆算実装

1. 完成形デモ体験 → 目標の明確化
2. HTML骨組み実装 → 機能要素の配置（見た目は気にしない）
3. CSS基本装飾 → 完成形と同じ見た目に
4. CSSアニメーション → ホバーエフェクト等の動きを追加
5. JavaScript機能 → 動的な操作を実現



詳細アジェンダ（120分）

Phase 0：完成形体験・分析（10分）

Step 0-1: デモ体験（5分）

- 完成形のタスク管理アプリを実際に操作
- チェックボックスをクリック、ホバー、進捗確認

Step 0-2: 機能分解（5分）

問いかけ：「今体験したサイトには、どんな要素・機能がありましたか？」

学習者に発見させる要素：

- ヘッダー（タイトル）
- 進捗バー
- 6つのタスクチェックボックス
- ホバーエフェクト
- 完了時のメッセージ表示

Phase 1：HTML構造実装（30分）

Step 1-1: 要素の洗い出し（10分）

問いかけ：「完成形を再現するために、HTMLでどんな要素が必要？」

思考プロセス:

- 完成形で見えた要素 → HTMLタグで表現
- タイトル部分 → header? h1?
 - 進捗バー → どうやって作る?
 - タスクリスト → input? label?
 - 完了メッセージ → div?

Step 1-2: HTML実装 (20分)

目標: 機能する骨組みを作る (見た目は一切気にしない)

実装戦略:

1. まずは必要最小限の構造
2. 後でCSSが当てやすいようにclass/id設計
3. セマンティックHTMLを意識

この段階の成果物:

```
<!-- 超シンプルな骨組み (装飾なし) -->
<header>
  <h1><img alt="book icon" data-bbox="185 430 205 445"/> 今日の学習タスク</h1>
  <p>目標: CSS装飾とJavaScript動きをマスターしよう!</p>
</header>

<section>
  <h2><img alt="progress bar icon" data-bbox="185 520 205 535"/> 学習進捗</h2>
  <div class="progress-bar">
    <div class="progress-fill"></div>
  </div>
  <p class="progress-text">0 / 6 完了 (0%)</p>
</section>

<!-- タスクリスト -->
<!-- 完了メッセージ -->
```

Phase 2: CSS基本装飾 (35分)

Step 2-1: 完成形との見た目比較 (5分)

問いかけ: 「今のHTMLと完成形、何が違う?」

ギャップ分析:

- 背景色
- カードのような白い背景
- 適切な余白・間隔
- 色使い
- フォント

Step 2-2: 基本レイアウトCSS (25分)

目標: 完成形と同じ見た目にする (アニメーションは除く)

段階的実装:

1. 全体設定 (5分)

```
* { margin: 0; padding: 0; box-sizing: border-box; }
body { background: #f3f4f6; font-family: sans-serif; }
```

2. カードレイアウト (10分)

```
.main-header, section {
  background: white;
  padding: 2rem;
  border-radius: 10px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
}
```

3. 進捗バーとタスクアイテム (10分)

Step 2-3: 完成形チェック (5分)

静的な見た目が完成形と一致しているか確認

Phase 3: CSSアニメーション (25分)

Step 3-1: アニメーション要件分析 (5分)

問いかけ: 「完成形では、どこでアニメーションが起きていた？」

発見すべきアニメーション:

- タスクアイテムのホバーエフェクト
- 進捗バーの幅変化
- 完了メッセージの表示

Step 3-2: ホバーエフェクト実装 (15分)

目標: マウスを乗せた時の反応を完成形と同じに

実装思考:

```
.task-item:hover {
  /* 何が変わってた? */
  /* どんな動きだった? */
}
```

段階的実装:

1. `:hover` の基本
2. `transition` でなめらかに
3. `transform` で浮遊感
4. `box-shadow` で影

Step 3-3: 動作確認 (5分)

完成形と同じアニメーションになっているかチェック

Phase 4: JavaScript動的機能 (20分)

Step 4-1: 動的機能の特定 (5分)

問いかけ: 「完成形で、クリックしたり操作すると何が起こっていた？」

発見すべき機能:

- チェックボックスをクリック → タスクの見た目変化
- チェックボックスをクリック → 進捗バー更新
- 全完了 → 完了メッセージ表示

Step 4-2: JavaScript実装 (15分)

目標: 完成形と同じ動的動作を実現

実装戦略:

1. 要素取得 (3分)

```
const checkboxes = document.querySelectorAll('.task-checkbox');
const progressFill = document.getElementById('progressFill');
```

2. イベント処理 (7分)

```
checkboxes.forEach(checkbox => {
  checkbox.addEventListener('change', function() {
    // ここで何をする?
  });
});
```

3. 進捗計算・表示 (5分)

各Phase間のチェックポイント

Phase間の振り返りルール

1. 「今何ができるようになった？」

2. 「完成形との差は何？」
3. 「次に実装すべきは何？」

ベクトル修正のサポート

- 各Phase開始時に完成形を再確認
- 実装中は完成形と並べて比較
- 「完成形では〇〇だったけど、今は△△だね」の指摘

エンジニア思考の体験

- 逆算思考: 「完成形を実現するには何が必要？」
- 段階的実装: 「まず動くものを作って、徐々に完成形に近づける」
- 比較検証: 「今の状態と目標の差分は何？」

このアプローチの利点は、**明確なゴール（完成形）**があることで**学習者が迷わない点**と、**実際の開発現場で使われる逆算・段階的実装の思考**を体験できる点です。